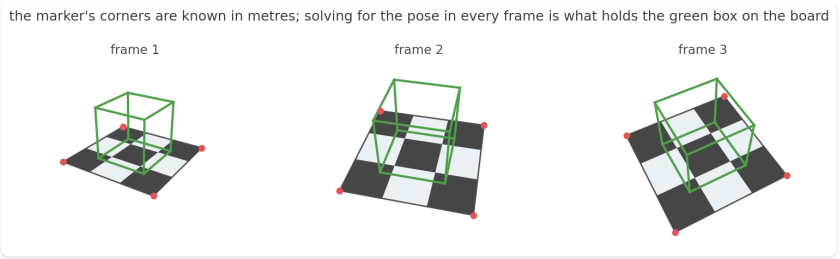
Industrial Vision

Lecture 13 - Where is the camera? The perspective-n-point problem

+ board

Where is the camera?

You print a marker whose corners you **know**, to the millimetre. You photograph it with a camera whose K you **know**, from last lecture.



From one photograph, can you say where you were standing, and which way you were looking?

This is today's lecture. Every AR app does it on every frame, and every visual-odometry and SLAM system does it against its map.

WHERE YOU WERE STANDING

translation t

3 numbers: a position

WHICH WAY YOU WERE LOOKING

rotation R

3 numbers: a direction and a roll

+ board

Last time vs today

	L14 calibration	**today**, PnP
K	unknown	**known**
3-D points	a flat board	**any** shape
views	several	**one**
we want	K	the pose $(R, \mathbf{t})$

Given world points $\mathbf{X}_i$, their pixels $\mathbf{x}_i$, and K.

Find R and $\mathbf{t}$ such that

$$\tilde{\mathbf{x}}_i \sim K[\,R \mid \mathbf{t}\,]\,\tilde{\mathbf{X}}_i$$

for every point i.

Zhang's method already did this for a **flat** board.
Today we drop the word *flat*.

+ board

How many points?

Unknowns	Each point gives	So we need
6	2	3
3 for rotation, 3 for position	its pixel (x_i, y_i)	points, since $2 \times 3 = 6$

> Six equations, six unknowns. Does that mean exactly **one** answer?

+ board

Method 1: DLT-PnP, the way we already know

Last lecture we found K by writing the unknowns as one vector, getting **linear** equations in it, and taking the null vector of a matrix. We do the same here. Put all of $K[R \mid \mathbf{t}]$ into one 3×4 matrix P and treat its **12 entries** as the unknowns:

$$\lambda_i \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

Here (X_i, Y_i, Z_i) is the known world point, (x_i, y_i) its pixel, and λ_i an unknown scale (the depth).

Write out the three rows:

$$\begin{aligned} \lambda_i x_i &= p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14} \\ \lambda_i y_i &= p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24} \\ \lambda_i &= p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34} \end{aligned}$$

The third row gives λ_i. Put it into the first two to remove it:

$$x_i = \frac{p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14}}{p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}}, \qquad y_i = \frac{p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24}}{p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}}$$

This is the homography of with one more column, the Z column, because the points no longer lie on a plane.

+ board

DLT-PnP: two linear equations per point

Multiply out the denominators. The equations become **linear** in the twelve p_{jk}:

$$p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14} - x_i\left(p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}\right) = 0$$
$$p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24} - y_i\left(p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34}\right) = 0$$

Collect the unknowns into $\mathbf{p} = (p_{11}, p_{12}, p_{13}, p_{14},\ p_{21}, \ldots, p_{24},\ p_{31}, \ldots, p_{34})^\top \in \mathbb{R}^{12}$. Each point gives **two rows**:

$$\begin{bmatrix} X_i & Y_i & Z_i & 1 & 0 & 0 & 0 & 0 & -x_iX_i & -x_iY_i & -x_iZ_i & -x_i \\ 0 & 0 & 0 & 0 & X_i & Y_i & Z_i & 1 & -y_iX_i & -y_iY_i & -y_iZ_i & -y_i \end{bmatrix} \mathbf{p} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Stack the rows of all n points into $A \in \mathbb{R}^{2n\times 12}$ and solve

$$A\,\mathbf{p} = \mathbf{0}, \qquad \|\mathbf{p}\| = 1 \quad \Longrightarrow \quad \mathbf{p} = \text{last right singular vector of } A$$

How many points? P has 12 entries but is known only up to scale, so 11 unknowns. Each point gives 2 equations, so $2n \geq 11$, that is $n \geq 6$.

+ board

DLT-PnP: from P to R and $\mathbf{t}$

The SVD returns P only up to an unknown scale μ (and sign). Since K is known, remove it:

$$M = K^{-1}P = \mu\,[\,R \mid \mathbf{t}\,] \qquad \Longrightarrow \qquad M_{3\times 3} = \mu R, \qquad \mathbf{m}_4 = \mu\,\mathbf{t}$$

- **Fix the sign first.** The null vector could equally be $-\mathbf{p}$, and $\det(-R) = -1$ for a 3×3 matrix. Since $\det(\mu R) = \mu^3 \det R = \mu^3$, the sign of $\det M_{3\times 3}$ is the sign of μ. If $\det M_{3\times 3} < 0$, multiply M by -1. Then $\mu > 0$, and the points lie in front of the camera.
- With noise, $M_{3\times 3}$ is **not** exactly μ times a rotation. Replace it by the nearest rotation, with the same SVD step as the per-view pose in **L14**:

$$M_{3\times 3} = U\Sigma V^\top, \qquad R = UV^\top$$

- The scale is the size of the block. For a true μR all three singular values equal μ, so take their mean: $\mu = \frac{1}{3}(\sigma_1 + \sigma_2 + \sigma_3)$, and then $\mathbf{t} = \mathbf{m}_4/\mu$.

> **The full recipe:** build A (2 rows per point), take its last right singular vector, reshape to P, compute $M = K^{-1}P$, make $\det M_{3\times 3} > 0$, and take the nearest rotation $UV^\top$. It needs **six** points.

+ board

Two questions about the DLT

Question 1. The count said a pose needs **3** points. The DLT needs **6**. Where did the other three points' worth of information go?

The DLT treats all 12 entries of P as **independent**. It does not use the fact that the left block is KR with R a **rotation**. A rotation has 9 entries but only 3 free numbers, so 6 constraints are left unused, and the DLT must make up for them with extra points. It is also less accurate for the same reason.

Question 2. Will the DLT work on the corners of a **flat** checkerboard?

No. Put the world frame on the board, so every point has $Z_i = 0$. Then p_{13}, p_{23}, p_{33} multiply zero in every row of A, so A says nothing about them. The system loses rank however many points you add. A flat target needs the planar (homography) solver of last lecture.

To use only three points, we must keep R a rotation from the start.

+ board

Method 2: a pixel is a direction

A pixel tells you the **ray** the point lies on, not how far along it.

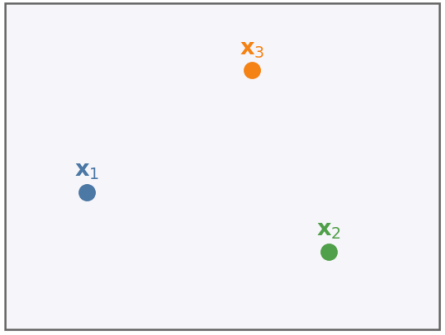
$$\mathbf{X}^c = d\,\mathbf{u}, \qquad \mathbf{u} = \frac{K^{-1}\tilde{\mathbf{x}}}{\|K^{-1}\tilde{\mathbf{x}}\|}$$

$\mathbf{X}^c = R\mathbf{X} + \mathbf{t}$ is the point in the camera frame, $\mathbf{u}$ is a unit vector along the ray, and $d > 0$ is the unknown distance.

Try it with $f = 500$ and centre $(320, 240)$:

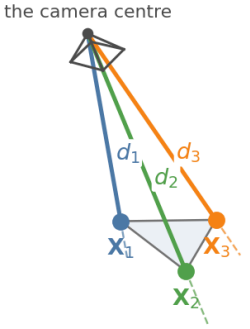
pixel	$K^{-1}\tilde{\mathbf{x}}$	angle off axis
$(320, 240)$	$(0, 0, 1)$	$0°$
$(820, 240)$	$(1, 0, 1)$	$45°$
same pixel, $f = 1000$	$(0.5, 0, 1)$	$26.6°$

what is measured: three points in the image



$\mathbf{u}_i = K^{-1}\tilde{\mathbf{x}}_i \,/\, \|K^{-1}\tilde{\mathbf{x}}_i\|$

what is unknown: three distances along known directions

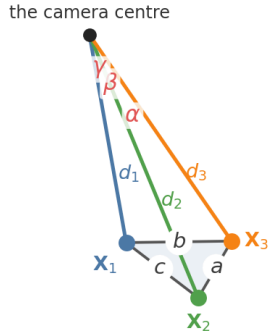


The angle between two rays is a dot product, $\cos\theta_{ij} = \mathbf{u}_i^\top \mathbf{u}_j$, computed from the pixels alone. **The unknowns are now three distances** d_1, d_2, d_3 along known rays.

+ board

P3P with the law of cosines

three known sides, three observed angles, three unknown distances



known: a, b, c from the world points, and $\cos\alpha = \mathbf{u}_2^\top \mathbf{u}_3$, $\cos\beta = \mathbf{u}_1^\top \mathbf{u}_3$, $\cos\gamma = \mathbf{u}_1^\top \mathbf{u}_2$ from the bearings. unknown: d_1, d_2, d_3

Grunert, 1841; named by Fischler and Bolles, 1981, in the paper that also introduced RANSAC.

Each pair of points and the camera centre form a triangle. We **know** its far side (the distance between the world points) and the angle at the camera (from the rays): $c^2 = d_1^2 + d_2^2 - 2\,d_1 d_2 \cos\gamma$. Therefore, the three pairs give **three quadratic equations** in d_1, d_2, d_3:

$$
\begin{aligned}
a^2 &= d_2^2 + d_3^2 - 2\,d_2 d_3 \cos\alpha \\
b^2 &= d_1^2 + d_3^2 - 2\,d_1 d_3 \cos\beta \\
c^2 &= d_1^2 + d_2^2 - 2\,d_1 d_2 \cos\gamma
\end{aligned}
$$

with $a = \|\mathbf{X}_2 - \mathbf{X}_3\|$, $\cos\alpha = \mathbf{u}_2^\top \mathbf{u}_3$, and so on.

+ board

P3P: up to four answers

- Substitute $d_2 = u\,d_1$ and $d_3 = v\,d_1$. Every term then has a factor d_1^2, which cancels when one equation is divided by another. This leaves two equations in u and v:

$$b^2(1 + u^2 - 2u\cos\gamma) = c^2(1 + v^2 - 2v\cos\beta), \qquad a^2(1 + v^2 - 2v\cos\beta) = b^2(u^2 + v^2 - 2uv\cos\alpha)$$

- Eliminating v from these two leaves one polynomial in u of **degree four**:

$$a_4u^4 + a_3u^3 + a_2u^2 + a_1u + a_0 = 0$$

The coefficients are known numbers, built from a, b, c and the three cosines, for example $a_4 = (a^2 - b^2 - c^2)^2 - 4b^2c^2\cos^2\alpha$ and $a_0 = (a^2 - b^2 + c^2)^2 - 4a^2c^2\cos^2\beta$. We do not write out a_1, a_2, a_3; the degree is what matters.

- A polynomial of degree four has up to **four** real roots. Each root gives u, then v, then d_1 from any of the three equations, so each root is a candidate set of distances.
- Discard roots that give a negative distance (a point behind the camera). Up to **four** valid poses can remain.

+ board

From distances to pose

Once d_1, d_2, d_3 are known, the points are known **in the camera frame**: $\mathbf{Y}_i = d_i \mathbf{u}_i$. We now have the same triangle in two frames, $\mathbf{X}_i$ in the world and $\mathbf{Y}_i$ in the camera.

1. Subtract the centroid of each triangle: $\mathbf{X}'_i = \mathbf{X}_i - \bar{\mathbf{X}}$, $\mathbf{Y}'_i = \mathbf{Y}_i - \bar{\mathbf{Y}}$.
2. $M = \sum_i \mathbf{Y}'_i \mathbf{X}'^{\top}_i = U \Sigma V^{\top}$.
3. $R = U \operatorname{diag}(1, 1, \det UV^{\top})\, V^{\top}$.
4. $\mathbf{t} = \bar{\mathbf{Y}} - R\bar{\mathbf{X}}$.

This is **Procrustes** from **L14**, the same SVD step we used for DLT-PnP in the previous slides. Here it finds the rotation that best lines up two point sets.

Why the $\det$ factor here? Three points always lie on a plane, so M has rank 2 and the sign of its third singular vector is arbitrary. Plain $UV^{\top}$ can then be a **reflection** ($\det = -1$). The factor flips that one direction and gives a rotation.

+ board

Real matches have outliers: RANSAC-PnP

1. **Sample** 4 matches.
2. **Solve** P3P on three; the 4th point picks the answer.
3. **Score**: count points that reproject within a few pixels.
4. **Repeat**, keep the best, and refit on its inliers.

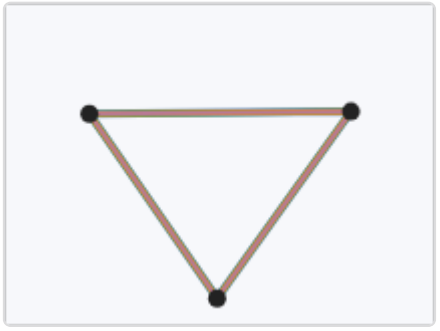
+ board

The fourth point: picking the right answer

A · the four solutions of P3P | B · RANSAC-PnP under outliers



four camera placements, three known points



the image they all produce

three points: all four poses reproject them exactly

pose	error on the 4th point
1	—
2	—
3	—
4	—

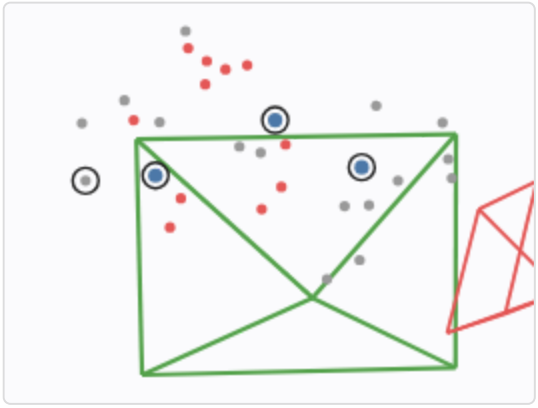
Three points give three quadratics, whose elimination is a quartic, so up to four poses explain the same three pixels. One more correspondence separates them, which is why implementations named for three points are handed four.

☐ use a fourth correspondence

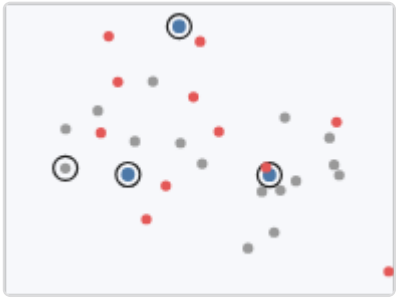
+ board

RANSAC-PnP at work

A · the four solutions of P3P | B · RANSAC-PnP under outliers



green: the true pose. red: the best pose so far, faint: this sample's



the 30 correspondences in the image

iteration 1: this sample supports **3** of 30; best so far **3**, against 18 true inliers

minimal sample	iterations for p = 0.99
s = 4, P3P	**33**
s = 6, DLT-PnP	96

At an inlier fraction of 0.6 the six-point solver needs 3 times as many iterations as the four-point one. The sample size sits in an exponent, which is the whole practical argument for a minimal solver.

outliers 40% iteration 1/24

Live: augmented reality on this webcam

- `1` P3P holds the object. The grey copy is the second answer.
- `2` DLT-PnP **fails** on the flat board, as predicted.
- `3` the planar solver of from last lecture holds.
- `o` scramble 40 % of the matches: only `4`, RANSAC, still holds.
- `k` wrong K: the object drifts, as in flatland.
- `--target box`: the points are no longer on one plane, and `2` now works.

key	solver
`1`	P3P, four corners
`2`	DLT-PnP, six corners
`3`	homography pose
`4`	RANSAC around P3P
`5`	`cv2.solvePnP`
`o` / `k`	outliers / wrong K
`m`	change model

The status line shows the distance to the board **in millimetres**, because the squares were printed at 25 mm.

+ board

Which solver, when?

situation	use
matches with outliers	**RANSAC** + **P3P** (sample of 4)
six or more clean points, not on one plane	**DLT-PnP**
a flat target	the **homography** pose
always, at the end	**refine** the pose by minimizing the reprojection error

> In practice these are combined: RANSAC with P3P to find the inliers, a linear solve on all the inliers, then refinement. Library calls such as `cv2.solvePnPRansac` package this pattern.

+ board

Why PnP gives metres

	data	minimal set	scale
homography H	image to image	4 points	not applicable
PnP	**world to image**	**3 points**	**metric**
essential E	image to image	5 points	**unknown**

PnP is metric because somebody **measured the world points in metres**, and the pose uses those measurements. Remove the known world and the scale is lost.

PnP and **triangulation** are opposite problems: known points and an unknown camera, against known cameras and an unknown point.

+ board

Questions

1. Why does DLT-PnP need six points when a pose has only six unknowns?

It solves for all 11 free entries of P and does not use the fact that R is a rotation. Two equations per point, so six points.

2. If P3P gives four answers, why not sample six points and use the DLT?

RANSAC's cost grows exponentially with the sample size, and the DLT also fails on planes and is less accurate.

3. Your AR cube drifts even with clean corners. What do you check first?

K (and the distortion). A wrong K gives wrong angles between the rays.

+ board

Summary

- **DLT-PnP**: write $P = K[R \mid \mathbf{t}]$ as 12 unknowns, get 2 linear equations per point, solve $A\mathbf{p} = \mathbf{0}$ by the SVD, then $K^{-1}P$ and the nearest rotation give R and $\mathbf{t}$. It needs **6** points and fails on a plane.
- It needs 6 points, not 3, because it does not use the fact that R is a rotation.
- A pixel is a **direction**. With K known, PnP becomes three **distances** along known rays.
- **P3P**: the law of cosines gives three quadratics and a degree-four polynomial, so up to **four** answers. A **fourth** point picks the right one.
- Distances $\rightarrow$ pose by **Procrustes**.
- **RANSAC + P3P** handles outliers. The answer is **metric** because the world points were.

+ board

References

1. Gao, Xiao-Shan, et al. "Complete solution classification for the perspective-three-point problem." IEEE transactions on pattern analysis and machine intelligence 25.8 (2003): 930-943.
2. R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge Univ. Press, 2004. See **§7.1** for the DLT for P and its degenerate configurations.

+ board